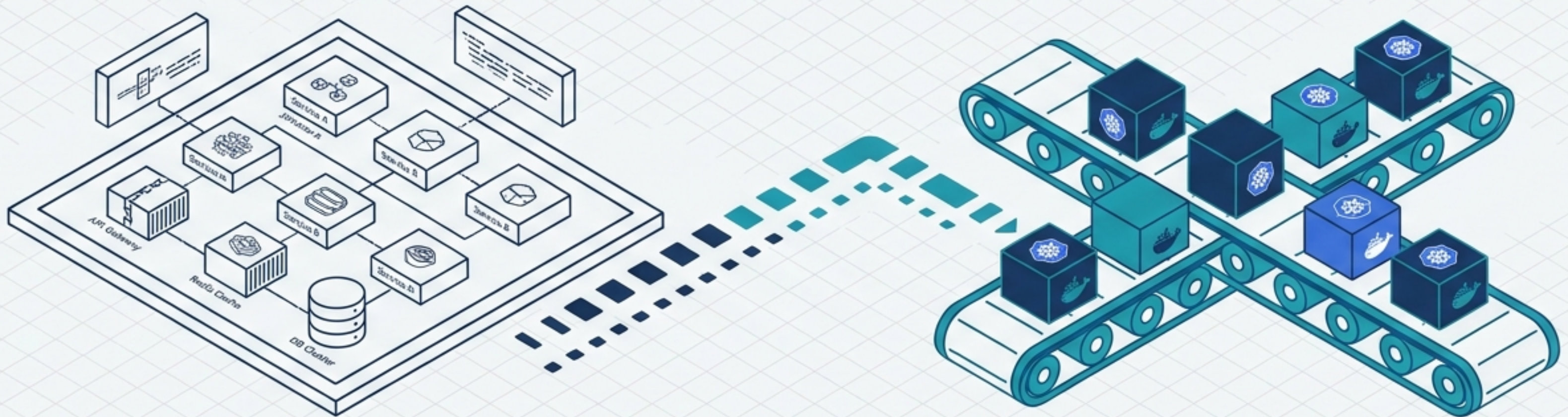


The Blueprint to the Assembly Line

Packaging and Orchestrating Kubernetes with Helm and Spinnaker

A high-level guide to modern deployment stacks, add-ons, and pipeline integration.



Helm acts as the package manager for Kubernetes platforms

Core Definition:

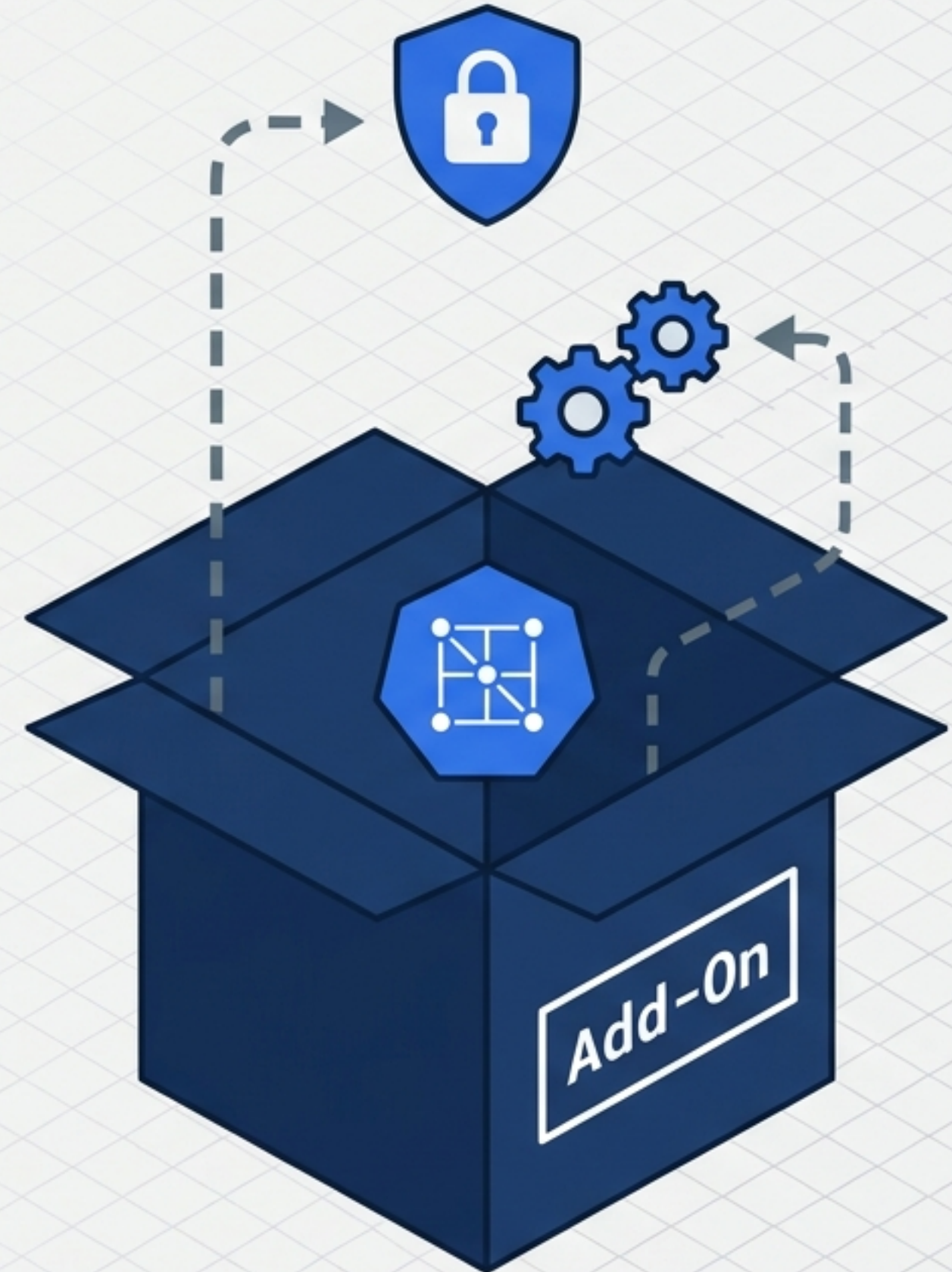
A Helm add-on is a reusable Kubernetes component packaged and deployed using Helm.

The Add-On Concept:

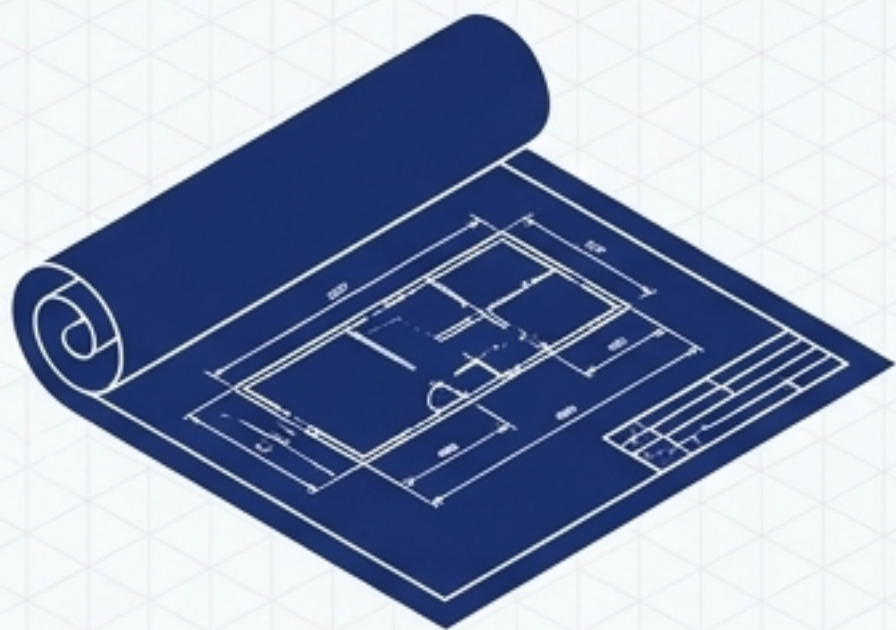
Platforms treat Helm charts as independent add-ons.

The Purpose:

An add-on owns a specific capability. It can be attached to any service or environment, completely eliminating the need for every individual service to duplicate Kubernetes YAML.



The anatomy of a Helm deployment



Chart

The package template.
Contains the core architectural definitions for Deployments, Services, ConfigMaps, and RBAC.



Values

Environment-specific configuration. Supplies exact parameters like image tags, replica counts, CPUs, URLs, and feature flags.



Release

One installed instance of that chart running live in a cluster.

Inside the structure of a Helm chart

```
my-addon/  
├── Chart.yaml  
├── values.yaml  
└── templates/  
    ├── deployment.yaml  
    ├── service.yaml  
    └── configmap.yaml
```

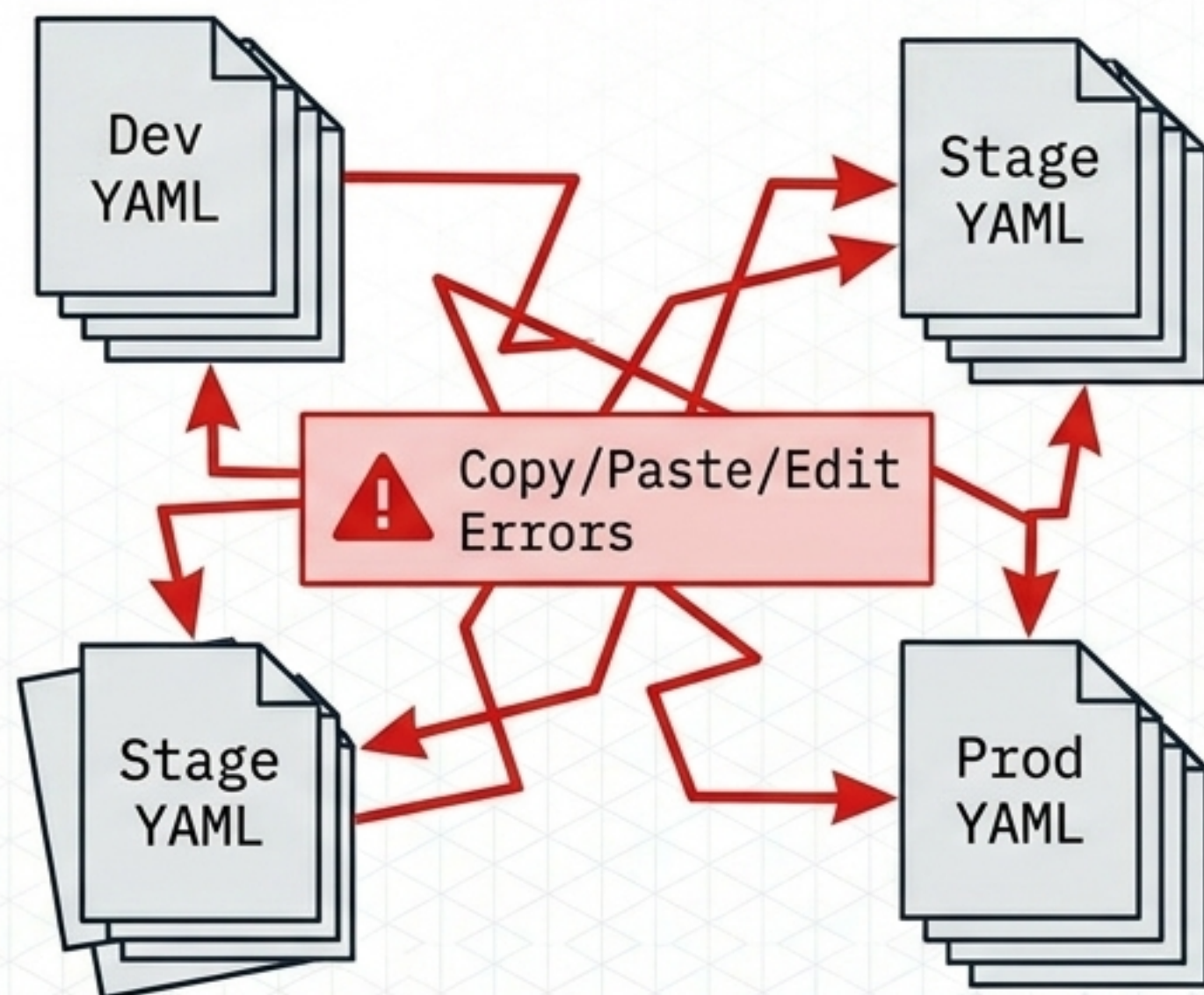
Defines the package metadata, chart name, version, and any dependencies.

Contains the default configuration values.

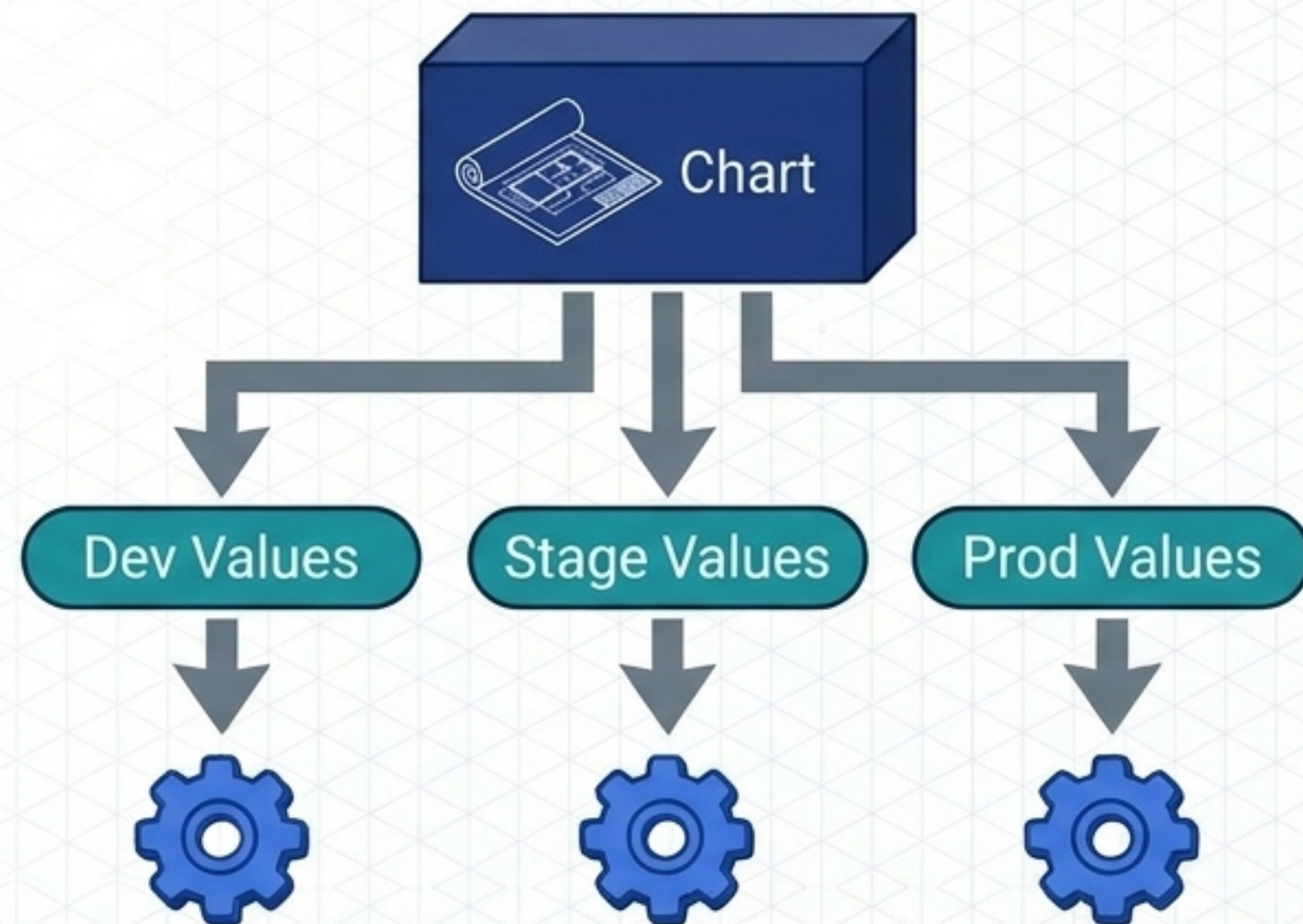
The core Kubernetes resource templates. These remain untouched by the consumer.

Eliminating YAML duplication across environments

Raw YAML



Helm Solution



Instead of copying and editing raw Kubernetes manifests for every environment, one Helm chart provides a consistent, versioned implementation. Consumers install the exact same chart version across all environments.

The four pillars of Helm add-ons



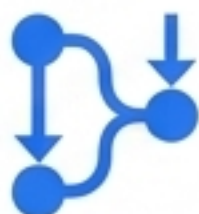
Reusable

A single chart can provision the exact same capability for dozens of different services simultaneously.



Configurable

Consumers change variables and values safely instead of rewriting or copying YAML structures.



Versioned

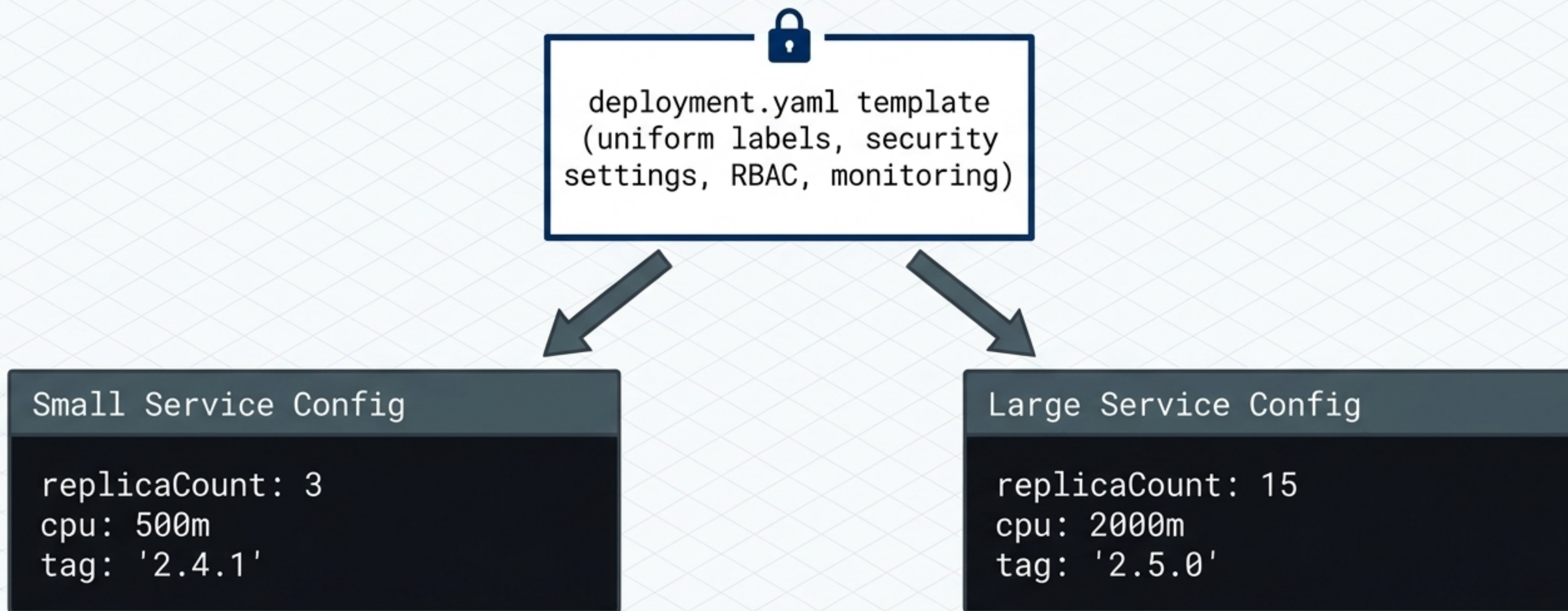
Chart versions make infrastructure upgrades and rollbacks highly controlled and predictable.



Lifecycle-Managed

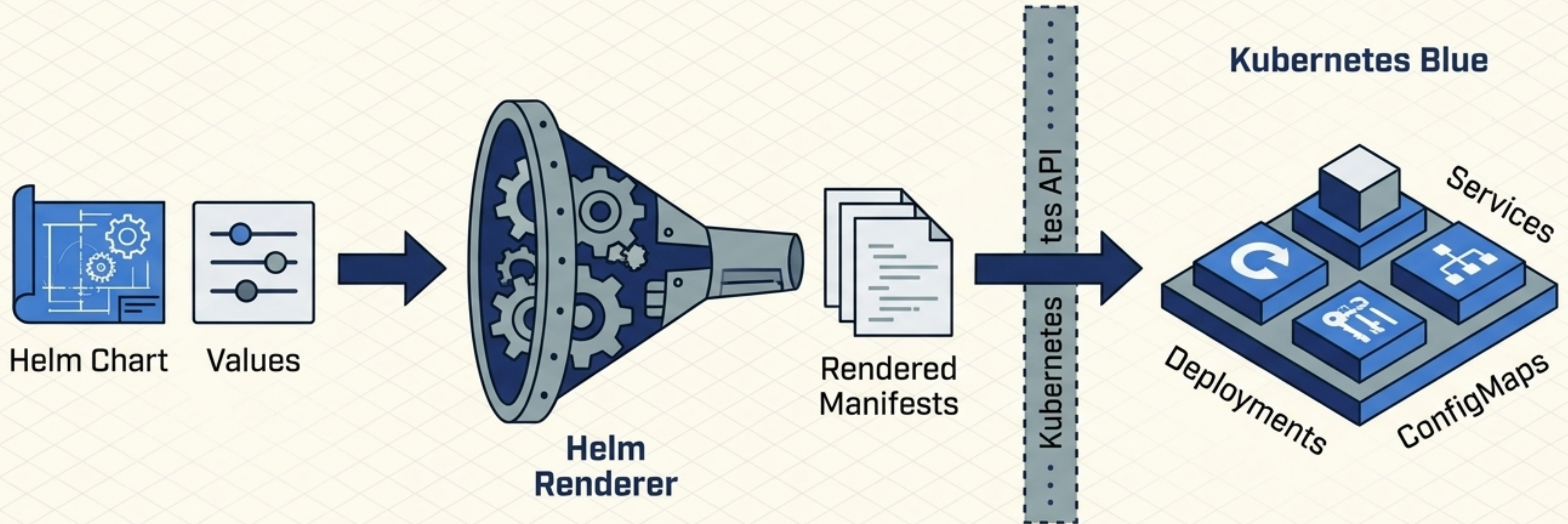
Core operations—install, upgrade, inspect, and uninstall—are standardized across all add-ons.

Safely varying installations via values.yaml



The chart keeps the complex architectural defaults uniform. Values let each installation vary safely without editing the underlying Kubernetes manifests.

Helm operates outside the Kubernetes runtime



- Helm is a part of the ecosystem, but it is not built into Kubernetes.
- Kubernetes runs the workloads. Helm renders the templates into manifests, then uses the Kubernetes API to apply them.

Executing a deployment

The standardized lifecycle command.

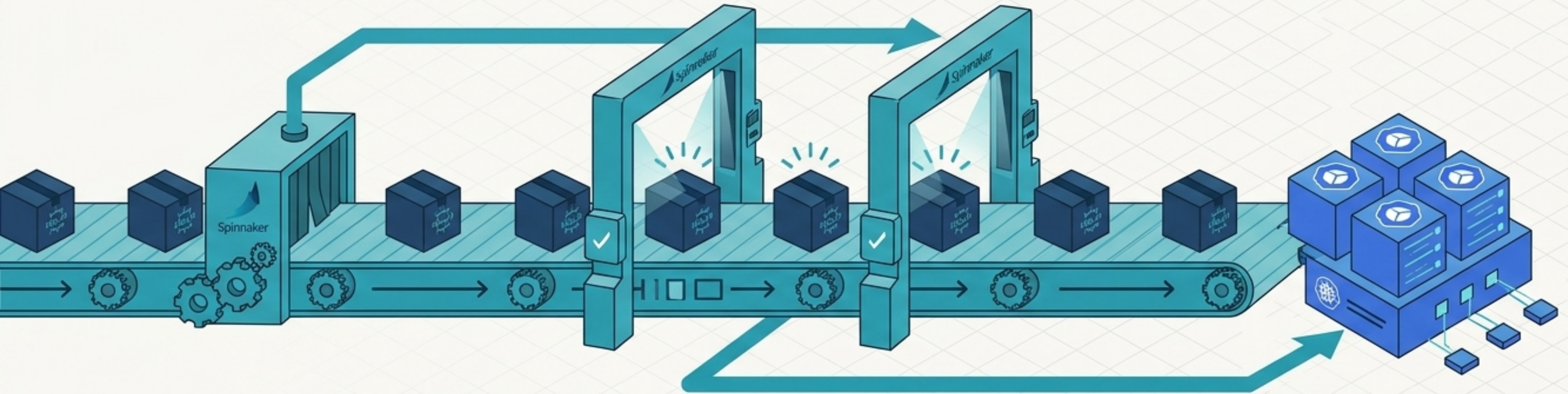
The Chart (the package template).

```
helm install redis-addon ./redis-chart  
--namespace my-service --set replicaCount=2
```

The specific Release name for this instance.

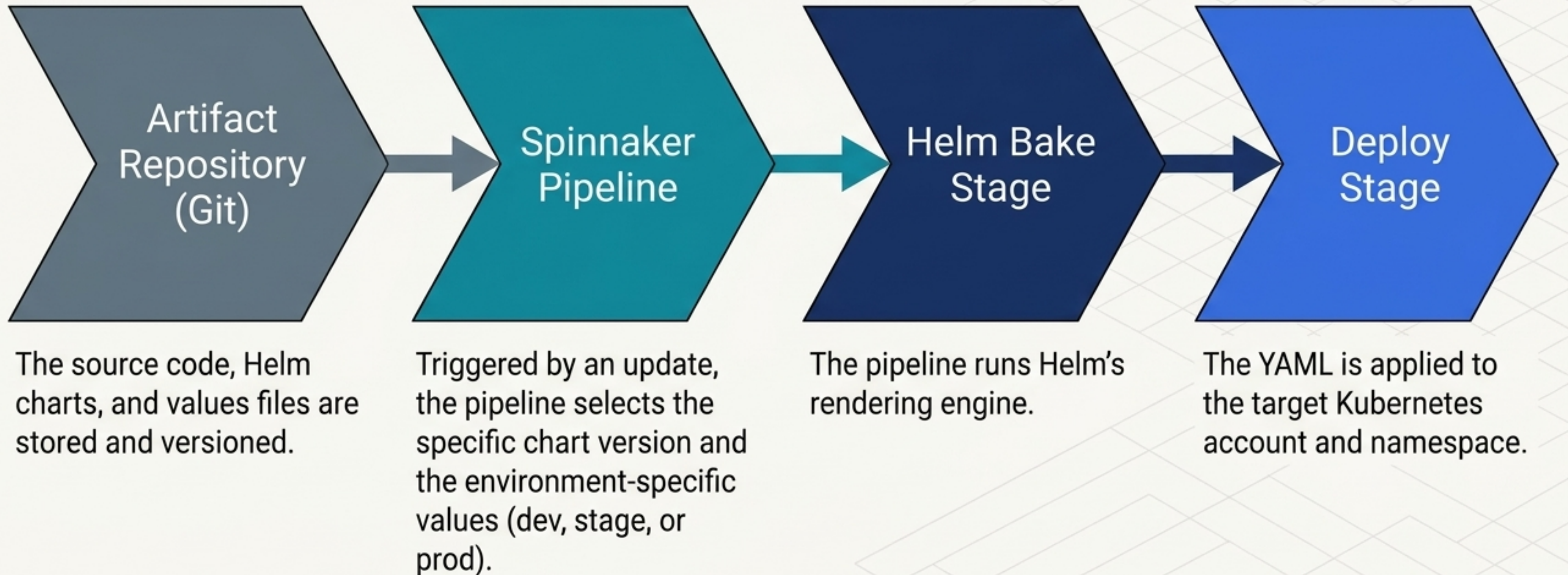
Supplying specific Values directly via CLI (overriding defaults).

Enter Spinnaker: The delivery orchestrator

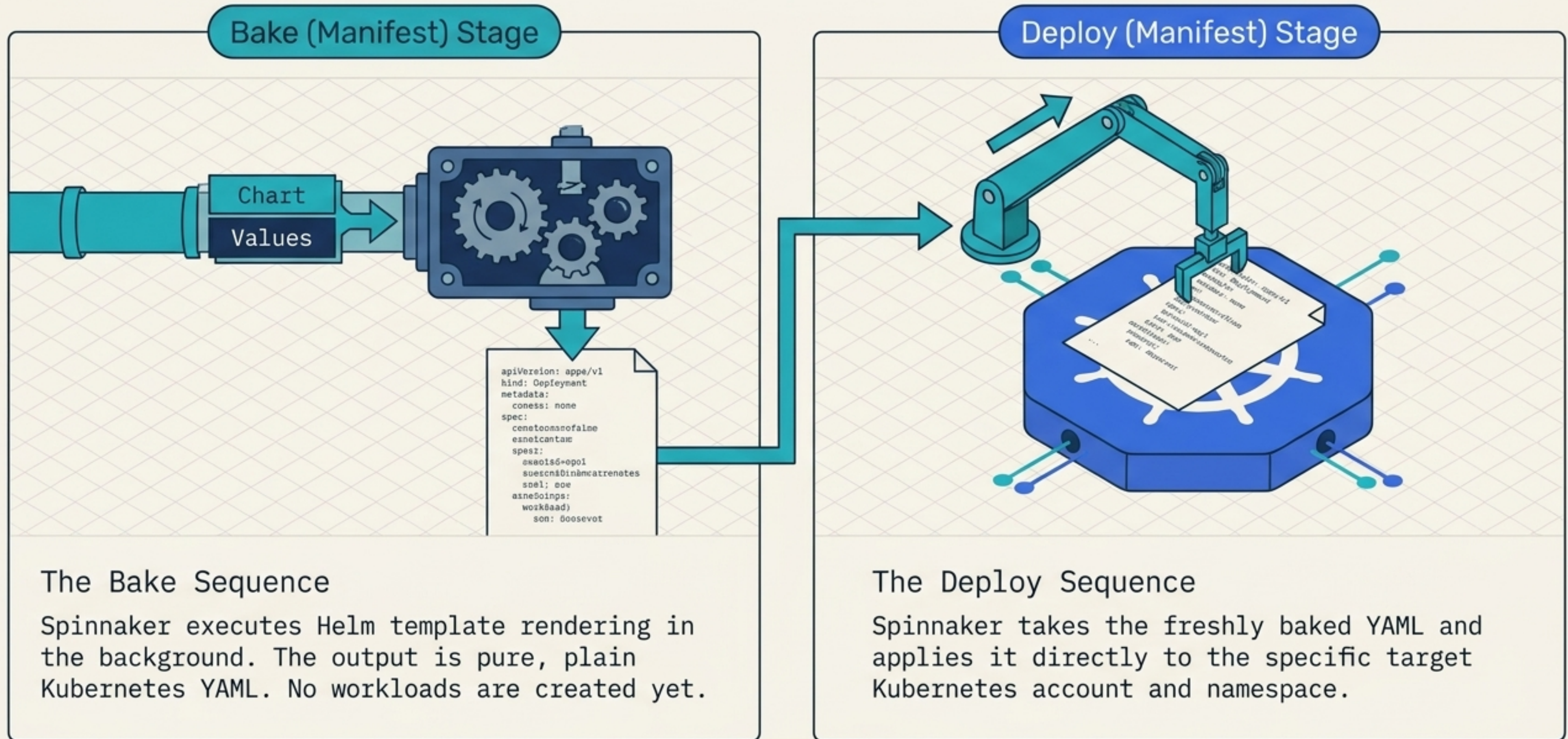


- While Helm solves packaging, enterprise platforms require complex pipeline delivery.
- Spinnaker is a dedicated deployment and orchestration system.
- Its native Helm integration allows a pipeline to safely and automatically deploy Helm charts to a target Kubernetes cluster.

The complete integration flow: Git to Cluster



The core Spinnaker mechanism: Bake vs. Deploy



The Orchestration Hand-off

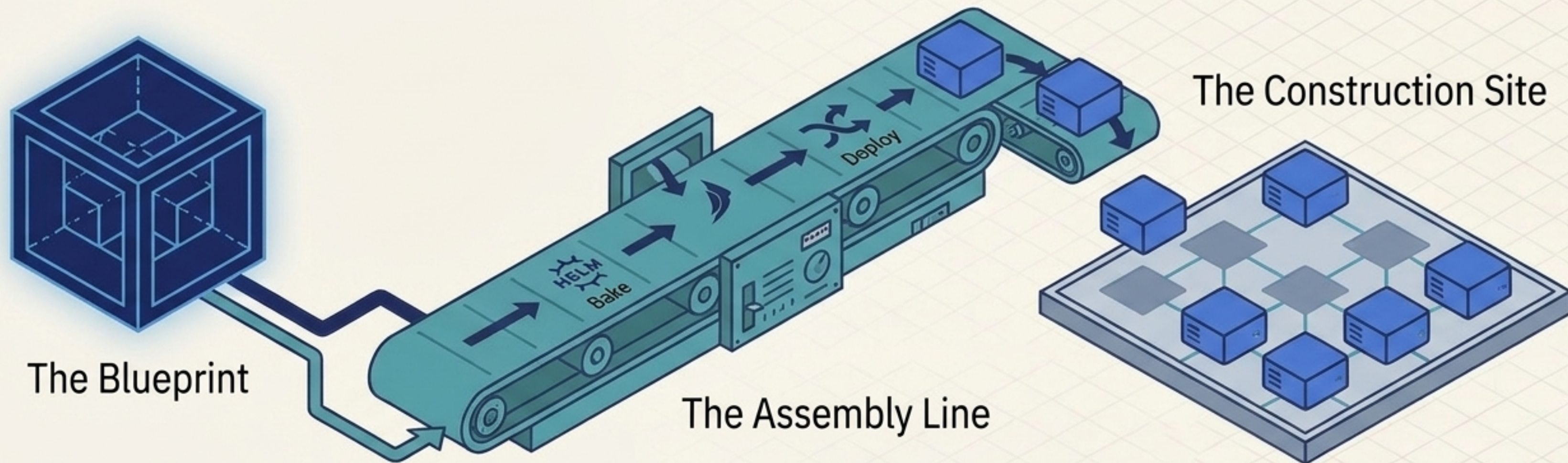


- Spinnaker uses Helm strictly as a rendering engine to generate the manifests.
- Once the YAML is baked, Helm's role is complete.
- Spinnaker takes over to manage the complex deployment workflow, tracking deployed resources, routing traffic, gating promotions with approvals, and providing visibility.

Separation of Concerns: The Ecosystem Matrix

	Helm	Spinnaker	Kubernetes
Primary Role	Package Manager & Renderer	Deployment Orchestrator	Runtime Workload Manager
Core Input	Templates & Values	Artifacts & Triggers	API Requests (YAML)
Core Output	Plain K8s YAML	Pipeline Workflows	Running Containers
Focus	Reusability & Configuration	Approvals, Routing & Rollbacks	Scheduling, State & Healing

Standardized packaging meets automated delivery



- A Helm add-on deploys shared service configurations safely and consistently.
- Spinnaker pipelines fully automate the retrieval, rendering, and complex rollout of those add-ons.
- Together, they decouple complex configuration from deployment execution.